

Research Methods

Day 8 · Chapter 7: Data Wrangling in Action

Outline for Today

1. **Detecting & Handling Missing Values**
2. **Identifying & Handling Outliers: The IQR Method**
3. **Standardizing Formats: Dates & Text**
4. **Filtering, Selecting & Combining Data**
5. **The 8-Step Data Cleaning Checklist**
6. **Coding Agents for Data Wrangling**



Chapter 7: Data Wrangling in Action

Learning Objectives

By the end of this chapter you will be able to:

1

Detect, evaluate, and handle missing values using listwise deletion and mean/median imputation — and explain which method is appropriate given the missingness type (MCAR vs. MNAR).

2

Apply the IQR method to detect outliers programmatically in both Python and R, and choose from four evidence-based handling strategies.

3

Standardize date formats and text inconsistencies using `pd.to_datetime()`, `str.strip()`, and regex replacement.

4

Filter, select, and reshape DataFrames using boolean indexing, `.query()`, and column selection.

5

Combine datasets using all four join types (inner, left, right, full) and vertical concatenation (append/bind_rows).

6

Apply the 8-step data cleaning checklist to a real messy dataset and document every decision in a cleaning log.

Detecting Missing Values – R

In R, `is.na()` returns TRUE for missing (NA) values and FALSE otherwise. Summing the result counts the total NAs, since TRUE = 1 and FALSE = 0 in R:

```
# Create a sample vector of scores with some NAs
scores <- c(88, 92, NA, 75, 90, NA, 85)
# Check which elements are NA
is.na(scores)
# [1] FALSE FALSE TRUE FALSE FALSE TRUE FALSE

# Count how many NAs
sum(is.na(scores))
# [1] 2
```

Detecting Missing Values – Python

In Python's pandas, `isnull()` (or equivalently `isna()`) returns a Series of booleans indicating missing values. We then use `.sum()` to count the True values:

```
import pandas as pd

# Create a pandas Series of scores with some NaN values
scores = pd.Series([88, 92, None, 75, 90, None, 85])
# Check which entries are null
scores.isnull()
# 0    False
# 1    False
# 2     True
# ...

# Count how many null values
scores.isnull().sum()
# 2
```

Two Ways to Handle Missing Data

Remove

Delete rows containing missing values (listwise deletion). Simple, but you lose data.

Impute

Fill in the blanks with an estimate — a static value, the mean, the median, or something smarter.

Removing Missing Data

- **Listwise deletion (complete-case analysis):** removes any row containing at least one missing value. Simple and guaranteed complete data — but valid only if data is **MCAR** (Missing Completely At Random).
- **Advanced:** `dropna(thresh=k)` keeps rows with at least k non-null values — useful when you can tolerate some missingness. `dropna(subset=['col'])` targets specific columns only.
- Simple and guarantees no missing values remain.
- **Downside:** if lots of data is missing, you lose a lot of cases.

Removing Missing Data

- **Listwise deletion (complete-case analysis):** removes any row containing at least one missing value. Simple and guaranteed complete data — but valid only if data is **MCAR** (Missing Completely At Random).
- **Advanced:** `dropna(thresh=k)` keeps rows with at least k non-null values — useful when you can tolerate some missingness. `dropna(subset=['col'])` targets specific columns only.
- Simple and guarantees no missing values remain.
- **Downside:** if lots of data is missing, you lose a lot of cases.

Critical: if missingness is MNAR (e.g., low scorers more likely to skip), listwise deletion inflates the apparent average. Always check whether completers differ from non-completers before deleting rows.

Removing Missing Data in R

The `drop_na` function in the `tidyr` package removes any rows with NA values:

Option 1

```
install.packages("tidyr") # If you haven't already installed it
library(tidyr)

# Suppose df is a data frame with some NAs
clean_df <- drop_na(df)
# This will drop any rows of df that contain NA in any column.
```

Option 2

```
clean_df <- na.omit(df)
```

Removing Missing Data in Python

Pandas provides the `dropna()` method for DataFrames:

```
# Assume df is a pandas DataFrame with some missing values  
clean_df = df.dropna()  
# By default, this drops any row with any NaN in it.
```

- `df.dropna(how='all')` would drop only rows that are **completely empty**.
- `df.dropna(subset=['col1', 'col2'])` would drop rows missing data in **specific columns** of interest.

Let's Do It!

1. Read the `students_scores.csv` data
2. Check the dimensions of the data
3. Remove missing students
4. Check the dimensions again
5. **How many rows did you lose?**

Solution

R

```
df <- read.csv("students_scores.csv")  
dim(df)  
  
df_clean <- na.omit(df)  
dim(df_clean)
```

Python

```
import pandas as pd  
  
df = pd.read_csv("students_scores.csv")  
df.shape  
  
df_clean = df.dropna()  
df_clean.shape
```

```
import pandas as pd
```

```
df = pd.read_csv("students_scores.csv")  
df.shape
```

(50, 4)

```
df_clean = df.dropna()  
df_clean.shape
```

(45, 4)

Imputing Missing Data (Filling in the Blanks)

Imputation replaces missing values with estimates — but every method makes assumptions. Mean imputation reduces variance and inflates correlations. *Choose the method based on why data is missing (MCAR/MAR/MNAR).*

Strategies from simple to advanced:

- Use a static value (e.g., 0 or "Unknown")
- Fill with the **mean or median** (for numeric data)
- For time-series: use forward fill (last value) or backward fill (next value)
- **MCAR data:** mean/median imputation is defensible. **MAR data:** regression imputation or KNN imputation. **MNAR data:** no simple fix — document and report the limitation.

Imputing Missing Data (Filling in the Blanks)

Imputation replaces missing values with estimates — but every method makes assumptions. Mean imputation reduces variance and inflates correlations. *Choose the method based on why data is missing (MCAR/MAR/MNAR).*

Strategies from simple to advanced:

- Use a static value (e.g., 0 or "Unknown")
- Fill with the **mean or median** (for numeric data)
- For time-series: use forward fill (last value) or backward fill (next value)
- **MCAR data:** mean/median imputation is defensible. **MAR data:** regression imputation or KNN imputation. **MNAR data:** no simple fix — document and report the limitation.

Mean imputation warning: replacing 20% of a variable with its mean artificially shrinks the standard deviation and inflates correlations with other variables. Always report the imputation method and percentage imputed.

Let's Practice with R

Option 1

```
scores <- c(88, 92, NA, 75, 90, NA, 85)
# Calculate mean of scores, excluding NA (na.rm = TRUE)
mean_score <- mean(scores, na.rm = TRUE)
mean_score
# [1] 86
# Replace NA values with the mean
scores_filled <- ifelse(is.na(scores), mean_score, scores)
scores_filled
# [1] 88.0 92.0 86.0 75.0 90.0 86.0 85.0
```

Option 2

```
library(tidyr)
scores_filled <- replace_na(scores, mean_score)
```

Let's Practice with Python

```
scores = pd.Series([88, 92, None, 75, 90, None, 85])
# Calculate mean of the scores (ignoring NaN by default)
mean_score = scores.mean()
mean_score
# 86.0

# Fill NaN values with the mean
scores_filled = scores.fillna(mean_score)
scores_filled
# 0      88.0
# 1      92.0
# 2      86.0
# 3      75.0
# 4      90.0
# 5      86.0
# 6      85.0
# dtype: float64
```

Identifying and Handling Outliers

❓ What Are Outliers?

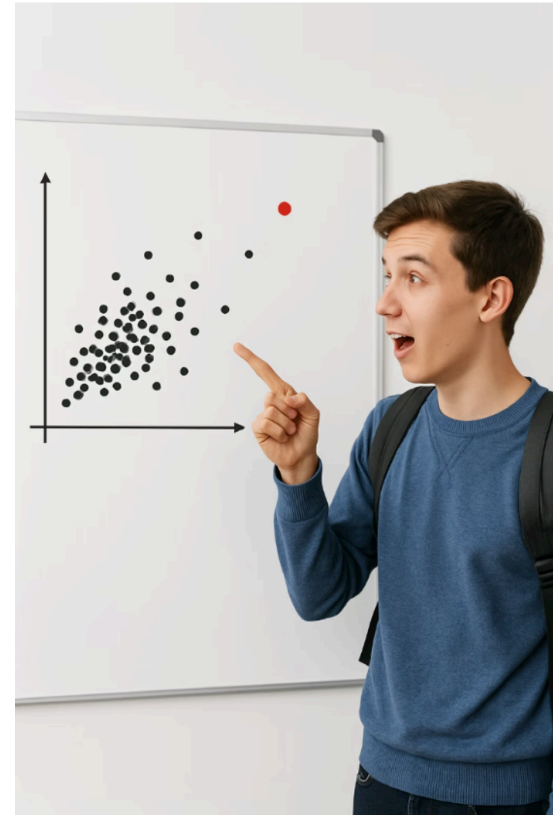
Values that deviate markedly from the rest of the distribution. **Important:** outliers are not inherently wrong — they may be legitimate extreme values (record-breaking athlete), data errors (typo), or evidence of a completely different process.

🔧 Examples

A student's height recorded as 300 cm or a survey response of "50" on a scale of 1–5.

⚖️ Why They Matter

A single outlier can shift the mean by several standard deviations while leaving the median unchanged. Comparing mean and median is



What Counts as an Outlier? – The IQR Method

The **interquartile range (IQR)** is the distance between the first and third quartiles:

$$\text{IQR} = Q_3 - Q_1$$

A value is flagged as an outlier if it falls outside the "fences":

$$x < Q_1 - 1.5 \times \text{IQR} \quad \text{or} \quad x > Q_3 + 1.5 \times \text{IQR}$$

What Counts as an Outlier? – The IQR Method

The **interquartile range (IQR)** is the distance between the first and third quartiles:

$$\text{IQR} = Q_3 - Q_1$$

A value is flagged as an outlier if it falls outside the "fences":

$$x < Q_1 - 1.5 \times \text{IQR} \quad \text{or} \quad x > Q_3 + 1.5 \times \text{IQR}$$

This is the same rule that box plots use to draw their whiskers – points beyond the whiskers are the outliers. The 1.5× multiplier is convention; 3× flags only "extreme" outliers.

Let's Practice with R

```
# Sample data: homework minutes
homework_minutes <- c(120, 90, 100, 95,
                     110, 105, 115, 300)
summary(homework_minutes)
# Min. 1st Qu. Median Mean 3rd Qu. Max.
# 90    95    105    129.4  115    300

# Calculate Q1, Q3, and IQR
Q1 <- quantile(homework_minutes, 0.25)
Q3 <- quantile(homework_minutes, 0.75)
IQR_val <- IQR(homework_minutes) # Q3 - Q1

# Determine the outlier thresholds
lower_thresh <- Q1 - 1.5 * IQR_val # 65
upper_thresh <- Q3 + 1.5 * IQR_val # 145

# Find values outside these thresholds
homework_minutes[homework_minutes < lower_thresh |
                  homework_minutes > upper_thresh]

# [1] 300
```

- In the example, $Q1 = 95$ and $Q3 = 115$, so the **IQR is 20**.
- Outlier thresholds: below **65** or above **145** minutes (using the $1.5 \times \text{IQR}$ rule).
- Most values fall within 65–145 minutes.
- One value, **300 minutes**, is flagged as an **outlier**.
- This could be a data entry error or a very unusual case.

Let's Practice with Python

```
import numpy as np

homework_minutes = np.array([120, 90, 100, 95, 110, 105, 115, 300])
Q1 = np.percentile(homework_minutes, 25)
Q3 = np.percentile(homework_minutes, 75)
IQR_val = Q3 - Q1
lower_thresh = Q1 - 1.5 * IQR_val
upper_thresh = Q3 + 1.5 * IQR_val
outliers = homework_minutes[(homework_minutes < lower_thresh) |
                             (homework_minutes > upper_thresh)]

print(Q1, Q3, IQR_val, lower_thresh, upper_thresh, outliers)
# 95.0 115.0 20.0 65.0 145.0 [300]
```

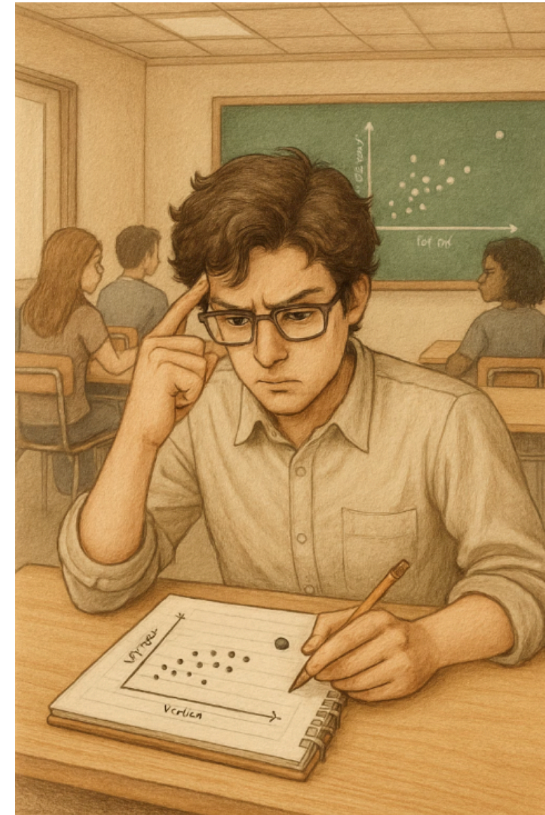
Handling Outliers: Four Strategies

1. Investigate Always investigate first: compare the outlier to the source data or domain knowledge. Is Age=300 physically impossible? Is Sales=\$50M plausible given other context? *The answer determines the correct strategy.*

2. Remove Remove only if confirmed data errors. Removing legitimate extreme values biases your analysis — the very student with 300 minutes of homework may be the most interesting data point.

3. Replace (Winsorize) Replace values above the 95th percentile with the 95th percentile value instead of removing them. Preserves the observation while limiting its influence:
`df["col"].clip(upper=df["col"].quantile(0.95))`

4. Keep but Use Robust Methods Sensitivity analysis: run the analysis both with and without the outlier. If conclusions change dramatically, report both results and let the reader decide.



Remove the Outlier with R

Option 1

```
# Remove outliers from the vector
homework_no_outliers <- homework_minutes[homework_minutes >= lower_thresh &
                                          homework_minutes <= upper_thresh]

homework_no_outliers
# [1] 120 90 100 95 110 105 115

mean(homework_minutes)
# [1] 129.375 (with the outlier)
mean(homework_no_outliers)
# [1] 105.0 (without the outlier)
```

Option 2

```
library(dplyr)
df_clean <- df %>% filter(minutes >= lower_thresh & minutes <= upper_thresh)
```

Remove the Outlier with Python

```
# Filter the numpy array for values within [lower_thresh, upper_thresh]
homework_no_outliers = homework_minutes[(homework_minutes >= lower_thresh) &
                                          (homework_minutes <= upper_thresh)]

print(homework_no_outliers)
# [120  90 100  95 110 105 115]

import pandas as pd
df = pd.DataFrame({'minutes': homework_minutes})
df_clean = df[(df['minutes'] >= lower_thresh) & (df['minutes'] <= upper_thresh)]
df_clean['minutes'].mean()
# 105.0
```

Standardizing Formats: Why It Matters

Date Format Issues

"2025-12-31" vs "12/31/2025" vs "31-Dec-2025" — all the same date!

Text Consistency Problems

"USA" vs "U.S.A." vs "United States" — all referring to the same country!

The Consequences

Inconsistent formats prevent matching, combining, and analyzing data correctly.



Fix Dates with R

The **lubridate** package (part of the tidyverse) is very handy for date parsing. First, install the package:

```
install.packages("lubridate")  
library(lubridate)
```

```
dates <- c("01/05/2022", "2022-01-06", "Jan 7 2022")  
# "January 5, 2022", "January 6, 2022", "January 7, 2022" in different formats.  
parsed_dates <- mdy(dates[1])           # mdy for "01/05/2022"  
parsed_dates <- c(parsed_dates, ymd(dates[2])) # ymd for year-month-day  
parsed_dates <- c(parsed_dates, mdy(dates[3])) # mdy for "Jan 7 2022"  
parsed_dates  
# [1] "2022-01-05" "2022-01-06" "2022-01-07"
```

Fix Dates with Python

```
import pandas as pd

dates = ["01/05/2022", "2022-01-06", "Jan 7 2022"]
parsed_dates = [pd.to_datetime(d) for d in dates]
print(parsed_dates)
# [Timestamp('2022-01-05 00:00:00'),
#   Timestamp('2022-01-06 00:00:00'),
#   Timestamp('2022-01-07 00:00:00')]
```

Fix Dates with Python

```
import pandas as pd

dates = ["01/05/2022", "2022-01-06", "Jan 7 2022"]
parsed_dates = [pd.to_datetime(d) for d in dates]
print(parsed_dates)
# [Timestamp('2022-01-05 00:00:00'),
#   Timestamp('2022-01-06 00:00:00'),
#   Timestamp('2022-01-07 00:00:00')]
```

Pandas recognized **all three formats** and converted them to Timestamp objects.

Text Consistency



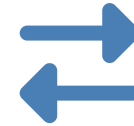
Case Consistency

Convert text to consistent case (upper, lower, or title case).



Remove Extra Spaces

Trim leading and trailing whitespace that can cause mismatches.



Standardize Synonyms

Map variations like "NYC," "New York City," and "New York, NY" to one standard form.

Text Standardization with R

Example 1

```
words <- c(" Apple", "apple ", "APPLE")  
# Trim whitespace  
words <- trimws(words)  
# Convert to lower case  
words_lower <- tolower(words)  
words_lower  
# [1] "apple" "apple" "apple"
```

Example 2

```
countries <- c("USA", "U.S.A.", "United States")  
countries_clean <- gsub("\\.|\\s", "", countries) # remove dots and spaces  
countries_clean <- ifelse(countries_clean %in% c("USA", "UnitedStates"),  
                          "USA", countries_clean)  
countries_clean  
# [1] "USA" "USA" "USA"
```

Text Standardization with Python

Example 1

```
words = pd.Series([" Apple", "apple ", "APPLE"])
words = words.str.strip()      # remove leading/trailing whitespace
words_lower = words.str.lower() # convert to lowercase
print(words_lower.tolist())
# ['apple', 'apple', 'apple']
```

Example 2

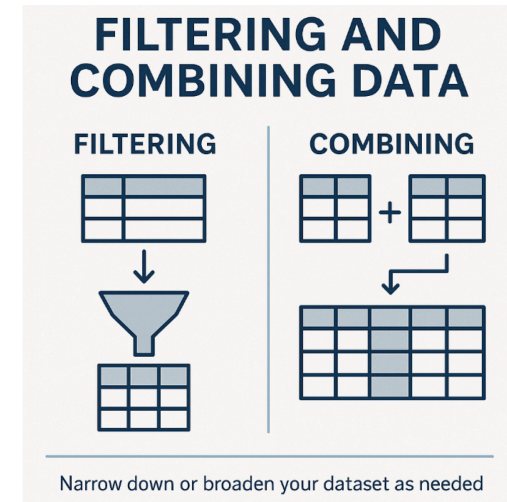
```
# For replacements and removing punctuation:
countries = pd.Series(["USA", "U.S.A.", "United States"])
countries_clean = countries.str.replace(r"\.", "", regex=True) # remove dots
countries_clean = countries_clean.str.replace(" ", "", regex=True) # remove spaces
countries_clean = countries_clean.replace({"United States": "USA"})
print(countries_clean.tolist())
# ['USA', 'USA', 'USA']
```

Filtering and Combining Data

- **Data subsetting is where analytical judgment and code meet:** filtering defines the population you're studying. Every filter is an analytical decision that should be documented — "I filtered to Grade==9" changes what conclusions are valid.
- **Method chaining:** in pandas, you can chain operations:

```
df.dropna().query("Grade==9")  
[["Student", "Math_Score"]].
```

 This produces clean, readable pipelines. In R, the pipe operator `|>` (or `%>%`) achieves the same.
- **Rule:** always filter *after* you've understood the full dataset. Premature filtering can hide patterns in data you haven't examined yet.



Filtering Rows by Condition Using R

```
library(dplyr)
# Assume students_df is a data frame with a column Grade (9, 10, 11, 12)
# We want only 9th graders
freshmen_df <- students_df %>% filter(Grade == 9)

# If we also only want 9th graders who scored above 80 in Math:
high_achievers <- students_df %>% filter(Grade == 9, MathScore > 80)
```

Filtering Rows by Condition Using Python

Option 1 – boolean indexing

```
df = pd.read_csv("students_scores.csv")
freshmen_df = df[df['Grade'] == 9]
# For multiple conditions, use & for AND, | for OR, with parentheses:
high_achievers = df[(df['Grade'] == 9) & (df['Math_Score'] > 80)]
```

Option 2 – .query()

```
freshmen_df = df.query("Grade == 9")
high_achievers = df.query("Grade == 9 and Math_Score > 80")
```

Let's Exercise

1. Read the `books.csv`
2. Filter data with **only mystery books**

R

```
books <- read.csv("books.csv")
mystery_books <- books %>%
  filter(Genre == "Mystery")
dim(mystery_books)
head(mystery_books)
```

Python

```
books = pd.read_csv("books.csv")
mystery_books = books[books['Genre'] == 'Mystery']
```

or:

```
> dim(mystery_books)
[1] 20  4
> head(mystery_books)
   Title  Genre  Author Year
1 Book_2 Mystery Author_1 1978
2 Book_3 Mystery Author_2 1991
3 Book_10 Mystery Author_9 1958
4 Book_14 Mystery Author_13 1977
5 Book_17 Mystery Author_16 2008
6 Book_18 Mystery Author_17 1968
```

Selecting and Dropping Columns with R

Selecting columns (or dropping columns) can make data easier to work with — and can **protect privacy** if we remove sensitive information:

```
# Select only Student, Grade, and Math_Score from df
small_df <- df %>% select(Student, Grade, Math_Score)
head(small_df)

# You can also drop columns by using a minus sign
no_names <- df %>% select(-Student, -English_Score)
head(no_names)
```

Selecting and Dropping Columns with Python

```
# Select only some columns
small_df = df[["Student", "Grade", "Math_Score"]]
small_df.head()

# Drop columns
no_names = df.drop(columns=["Student", "English_Score"])
no_names.head()
```

```
# To drop columns, you can use the drop method:
no_names = df.drop(columns=['Student', 'English_Score'])
no_names.head()
```

	Math_Score	Science_Score	Grade
0	88	51.0	9
1	78	NaN	11
2	64	77.0	11
3	92	96.0	11
4	57	56.0	10

Combining Data: Merging and Joining

Merging Tables

Combining data from different sources based on a common key.

Matching Keys

Finding records that match between datasets.

Appending Rows

Stacking datasets with the same structure.

Verifying Results

Ensuring the combined data makes sense.

Combining Data with R

```
# Filter Student Scores
df <- read.csv("students_scores.csv")
scores_df <- df %>% select("Student", "Math_Score", "English_Score")

# Read Student Information
info_df <- read.csv("Student_Information.csv")
combined_df <- scores_df %>% left_join(info_df, by = "Student")
head(combined_df)
```

Combining Data with R

```
# Filter Student Scores
df <- read.csv("students_scores.csv")
scores_df <- df %>% select("Student", "Math_Score", "English_Score")

# Read Student Information
info_df <- read.csv("Student_Information.csv")
combined_df <- scores_df %>% left_join(info_df, by = "Student")
head(combined_df)
```

```
> head(combined_df)
```

	Student	Math_Score	English_Score	Name	Grade	Address
1	Student_1	88	58	Riley Wilson	9	872 Elm St, Franklin, GA
2	Student_2	78	73	Drew Lopez	11	337 Willow Way, Georgetown, NC
3	Student_3	64	50	Morgan Lee	11	779 Willow Way, Riverside, TX
4	Student_4	92	93	Jamie Brown	11	461 Birch Blvd, Georgetown, IL
5	Student_5	57	57	Casey Gonzalez	10	577 Birch Blvd, Salem, FL
6	Student_6	70	73	Riley Lee	9	284 Chestnut Pl, Clinton, PA

The Four Join Types

LEFT JOIN: all rows from the left table, matched rows from the right. NaN fills unmatched right-side values. *Use when the left table is your analytical ground truth and you need to enrich it.*

INNER JOIN: only rows present in **both** tables. *Warning: silently drops unmatched rows – check `df.shape` before and after to confirm no unexpected data loss.*

FULL JOIN: all rows from both tables, NaN where no match. *Use for data reconciliation: finding records in one source but not another.*

RIGHT JOIN: all rows from the right table, matched rows from the left. Equivalent to a left join with the tables swapped.

The Four Join Types

LEFT JOIN: all rows from the left table, matched rows from the right. NaN fills unmatched right-side values. *Use when the left table is your analytical ground truth and you need to enrich it.*

INNER JOIN: only rows present in **both** tables. *Warning: silently drops unmatched rows – check `df.shape` before and after to confirm no unexpected data loss.*

FULL JOIN: all rows from both tables, NaN where no match. *Use for data reconciliation: finding records in one source but not another.*

RIGHT JOIN: all rows from the right table, matched rows from the left. Equivalent to a left join with the tables swapped.

After any join: check `df.shape` and `df.isnull().sum()` to verify.

Combining Data with Python

```
df = pd.read_csv("students_scores.csv")
scores_df = df[["Student", "Math_Score", "English_Score"]]

info_df = pd.read_csv("Student_Information.csv")
combined_df = pd.merge(scores_df, info_df, on="Student", how="left")

combined_df.head()
```

Combining Data with Python

```
df = pd.read_csv("students_scores.csv")
scores_df = df[["Student", "Math_Score", "English_Score"]]

info_df = pd.read_csv("Student_Information.csv")
combined_df = pd.merge(scores_df, info_df, on="Student", how="left")

combined_df.head()
```

:	Student	Math_Score	English_Score	Name	Grade	Address
0	Student_1	88	58	Riley Wilson	9	872 Elm St, Franklin, GA
1	Student_2	78	73	Drew Lopez	11	337 Willow Way, Georgetown, NC
2	Student_3	64	50	Morgan Lee	11	779 Willow Way, Riverside, TX
3	Student_4	92	93	Jamie Brown	11	461 Birch Blvd, Georgetown, IL
4	Student_5	57	57	Casey Gonzalez	10	577 Birch Blvd, Salem, FL

Appending Rows (Stacking Data)

❓ What is Appending?

Stacking one dataset below another with the **same columns** — like adding more pages to a book with the same format.

☰ Common Scenarios

- Combining data from different time periods
- Merging data from different locations
- Consolidating multiple surveys with the same questions
- Building a complete dataset from partial files

Appending Rows with R

```
# Read the 2021 scores CSV file into a data frame
scores_2021_df <- read.csv("Scores_2021.csv")
dim(scores_2021_df)

# Read the 2022 scores CSV file into a data frame
scores_2022_df <- read.csv("Scores_2022.csv")
dim(scores_2022_df)

# Combine the 2021 and 2022 data frames by stacking them
all_scores_df <- bind_rows(scores_2021_df, scores_2022_df)

# Display the dimensions of the combined data frame
dim(all_scores_df)
```

Appending Rows with Python

```
# Read the 2021 scores CSV file into a DataFrame
scores_2021_df = pd.read_csv("Scores_2021.csv")
print(scores_2021_df.shape)

# Read the 2022 scores CSV file into a DataFrame
scores_2022_df = pd.read_csv("Scores_2022.csv")
print(scores_2022_df.shape)

# Combine the 2021 and 2022 DataFrames by stacking them
all_scores_df = pd.concat([scores_2021_df, scores_2022_df],
                           ignore_index=True)
print(all_scores_df.shape)
```

Your Turn!

We have 2 datasets: **lab_results.csv** and **patients.csv**

1. Combine the two files
2. Experiment with different join types

It's Your
Turn!

Answers with R: Read and Know Your Data

```
# 1. Read the DataSets
lab_results <- read.csv("lab_results.csv")
patients <- read.csv("patients.csv")

# 2. Know your data
# Check the heads
head(lab_results)
head(patients)

# Check the dimensions
dim(lab_results) # 35 3
dim(patients)    # 30 3
```

```
> head(lab_results)
  ID      Test Result
1 21      X-Ray   Clear
2  4 Allergy Test Negative
3  1 Allergy Test     Low
4 24 Blood Sugar Negative
5  9 Allergy Test     High
6  8 Allergy Test   Clear

> head(patients)
  ID  Name Age
1  1  Alice 58
2  2   Bob 48
3  3 Charlie 34
4  4  Diana 62
5  5 Edward 27
6  6  Fiona 40

> # Check the dimensions
> dim(lab_results) # 35 3
[1] 35  3
> dim(patients) # 30 3
[1] 30  3
```

Answers with R: inner_join and full_join

Try inner_join — it kept all the matching data:

```
# 3. Try inner_join
library(dplyr)
in_join_df <- patients %>%
  inner_join(lab_results, by = "ID")

dim(in_join_df)
head(in_join_df)
```

Try full_join — it kept all data regardless of match:

```
# 4. Try full_join
f_join_df <- patients %>%
  full_join(lab_results, by = "ID")

dim(f_join_df)
head(f_join_df)
tail(f_join_df) # patients without lab results
```

```
> dim(f_join_df) #kept all the lab_results and patients
[1] 46 5
> head(f_join_df)
  ID Name Age Test Result
1  1 Alice 58 Allergy Test Low
2  1 Alice 58 Blood Sugar Low
3  1 Alice 58 Allergy Test Low
4  2 Bob 48 MRI Clear
5  3 Charlie 34 X-Ray Positive
6  3 Charlie 34 X-Ray High
> tail(f_join_df) #kept patients without the lab results
  ID Name Age Test Result
41 25 Yara 52 <NA> <NA>
42 26 Zane 31 <NA> <NA>
43 27 Liam 41 <NA> <NA>
44 28 Maya 63 <NA> <NA>
45 29 Noah 44 Cholesterol High
46 30 Olivia 68 <NA> <NA>
```

Answers with Python

```
# 1. Read the DataSets
import pandas as pd
lab_results = pd.read_csv("lab_results.csv")
patients = pd.read_csv("patients.csv")

# 2. Know your data
print(lab_results.head())
print(patients.head())
print(lab_results.shape)
print(patients.shape)

# 3. Try inner join
in_join_df = pd.merge(patients, lab_results, on="ID", how="inner")
print(in_join_df.shape)

# 4. Try outer (full) join – same as full_join in R
full_join_df = pd.merge(patients, lab_results, on="ID", how="outer")
print(full_join_df.shape)
```



Chapter 7 Summary: The Data Cleaning Checklist

Data Cleaning Checklist (1/2)

1

Understand Your Data Context

Read the data dictionary first. Without domain knowledge, you can't distinguish a valid extreme value from an error. Ask: what is the unit? What is the valid range? What do missing values mean in this context?

2

Check the Structure

Check `df.shape`, `df.dtypes`, `df.describe()`, `df.isnull().sum()` for every new dataset. Before cleaning anything, form hypotheses about what you expect to find — surprises are the most important signals.

3

Handle Missing Data

Identify missingness type (MCAR/MAR/MNAR). Choose method: listwise deletion only for MCAR; mean/median imputation for continuous MCAR/MAR; document all decisions with before/after row counts.

4

Identify Outliers

Apply at least two detection methods (IQR + domain knowledge). For each flagged value: investigate before acting. Document whether you removed, replaced, winsorized, or kept it.

Data Cleaning Checklist (2/2)

5

Standardize Formats

Parse all dates to a single format (ISO 8601: YYYY-MM-DD) using `pd.to_datetime()`. Normalize text with `.str.strip().str.lower()`. Test with `value_counts()` after each transformation.

6

Filter as Needed

Every filter is an analytical decision — document it in your log. "Filtered to Grade==9" restricts generalizability. If you filter outliers, note that conclusions apply only to the filtered population.

7

Combine Data

Always check row counts after merging. An unexpected row count increase suggests many-to-many matches. An unexpected decrease indicates mismatched keys. Both are data quality issues, not just code bugs.

8

Verify and Save Clean Data

Run your original `df.describe()` and `df.isnull().sum()` against the cleaned version. Also run basic sanity checks: are there still negative ages? Any dates in the future? Does the mean match domain expectations?

Let's Merge and Clean!

You have two datasets:

- school_participation_class1.csv
- school_participation_class2.csv

Columns: StudentID, Name, Grade, Age, Club, SignupDate.

Upon inspection, you find:

- Some Age values are **missing**.
- One Age is recorded as **144** (clearly an error, likely meant 14).
- The SignupDate column has dates in **mixed formats**: some are "MM/DD/YYYY", others are "Month Day, Year".
- Grade is recorded as 9, 10, 11, 12 but one entry says "Freshman" instead of 9

Your task: use the checklist to clean this data.

1. Understand Your Data Context
2. Check the Structure
3. Handle Missing Data
4. Identify Outliers
5. Standardize Formats
6. Filter as Needed
7. Combine Data
8. Verify and Save Clean Data

Answers

 Check one way to deal with these issues in R: `Ch7_FinalTaskAnswer.R`

 For Python: `Ch7_FinalTaskAnswer.ipynb`

 Check the example log record: `Ch7_FinalTask_data_cleaning_log.csv`

Answers

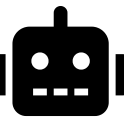
 Check one way to deal with these issues in R: `Ch7_FinalTaskAnswer.R`

 For Python: `Ch7_FinalTaskAnswer.ipynb`

 Check the example log record: `Ch7_FinalTask_data_cleaning_log.csv`

Key principle: there is no single correct cleaning approach — different analytical questions may justify different decisions. What matters is that **every decision is documented and defensible**.

Your cleaning log is as important as the cleaned data. Without it, your analysis is not reproducible. A cleaned dataset without a log cannot be verified, audited, or extended by collaborators.



Coding Agents for Data Wrangling

Coding Agent Workflow for Data Wrangling

A coding agent can automate the full 8-step cleaning pipeline — **but your analytical judgment drives every decision:**

✗ **Weak:** "Clean this dataset." → Agent applies arbitrary defaults (e.g., drops all rows with any missing value) without knowing your analytical goals.

✓ **Strong:** "Load school_participation_class1.csv and class2.csv. Stack them with `pd.concat`. Then: (1) report `df.shape` and `df.dtypes`, (2) count missing values per column and flag rates >5%, (3) apply IQR outlier detection to all numerical columns, (4) report all unique values in the `Grade` and `SignupDate` columns."

After the audit, give cleaning instructions one step at a time:

- "Fix SignupDate: convert all values to ISO format YYYY-MM-DD using `pd.to_datetime()`. Show me the first 5 rows of SignupDate before and after."
- "Standardize Grade: replace 'Freshman' with 9. Show `value_counts()` before and after."
- "After every cleaning step, print `df.shape` so I can verify no rows were unexpectedly dropped."

Verifying Agent-Cleaned Data

The most important skill: knowing when the agent made a plausible-but-wrong decision.

Common agent mistakes in wrangling:

1. Drops rows with ANY missing value when you only needed to handle one column.
2. Converts a ZIP code (stored as "06001") to integer 6001 — destroying the leading zero.
3. Fills all missing Ages with the mean, including rows where Age was MNAR.

Verify after every step: compare `df.shape` before and after. Check `df.dtypes` to catch silent type conversions. Run `value_counts()` on categorical columns to confirm standardization worked.

Verifying Agent-Cleaned Data

The most important skill: knowing when the agent made a plausible-but-wrong decision.

Common agent mistakes in wrangling:

1. Drops rows with ANY missing value when you only needed to handle one column.
2. Converts a ZIP code (stored as "06001") to integer 6001 — destroying the leading zero.
3. Fills all missing Ages with the mean, including rows where Age was MNAR.

Verify after every step: compare `df.shape` before and after. Check `df.dtypes` to catch silent type conversions. Run `value_counts()` on categorical columns to confirm standardization worked.

The cleaning log is your verification tool:

- Ask: "Produce a cleaning log as a numbered list. For each step: what was changed, how many values/rows were affected, and what method was used."
- Then verify each log entry against the data: if the log says "3 duplicate rows removed," confirm with `df.shape` that you lost exactly 3 rows, not 3 different ones.
- **Final check:** rerun your original data quality audit on the cleaned dataset. Every issue from Step 1 should now be resolved.

Hands-On: Agent-Assisted Wrangling of Class Participation Data

Activity (pairs, 25 minutes) — using `school_participation_class1.csv` and `class2.csv`:

1

Stack and audit: Prompt the agent to concatenate the two files and produce a full data quality report (shape, dtypes, missing rates, unique values per column).

2

Clean one issue at a time: Direct the agent to fix Age=144, standardize date formats, fix Grade="Freshman", and remove duplicates. After each fix, verify with a targeted check (not a general "looks good").

3

Impute missing Ages: Ask the agent to impute with the column mean. Then ask: "How many rows were imputed? What was the mean before and after? Did the standard deviation change?" Explain why it changed.

4

Generate the cleaning log: Ask for a numbered log matching the professional format from Chapter 6. Compare your log to the example answer.

Hands-On: Agent-Assisted Wrangling of Class Participation Data

Activity (pairs, 25 minutes) — using `school_participation_class1.csv` and `class2.csv`:

1

Stack and audit: Prompt the agent to concatenate the two files and produce a full data quality report (shape, dtypes, missing rates, unique values per column).

2

Clean one issue at a time: Direct the agent to fix Age=144, standardize date formats, fix Grade="Freshman", and remove duplicates. After each fix, verify with a targeted check (not a general "looks good").

3

Impute missing Ages: Ask the agent to impute with the column mean. Then ask: "How many rows were imputed? What was the mean before and after? Did the standard deviation change?" Explain why it changed.

4

Generate the cleaning log: Ask for a numbered log matching the professional format from Chapter 6. Compare your log to the example answer.

Challenge: find one issue the agent misses or handles incorrectly. Document it and explain the better approach.

Chapter 7: What You Can Do Now

Missing data handling:

- Detect missing values with `is.na()/isnull().sum()`. Choose listwise deletion vs. mean/median imputation based on missingness type (MCAR/MAR/MNAR). Report imputation method and percentage imputed.

Outlier detection and treatment:

- Apply IQR outlier detection in Python (`numpy.percentile`) and R (`quantile/IQR`). Choose from the four strategies (investigate, remove, winsorize, robust methods) based on whether the outlier is an error or a legitimate extreme value.

Data transformation and combination:

- Standardize dates with `pd.to_datetime()` and text with `.str.strip()/ .str.lower()/regex`. Filter with boolean indexing and `.query()`. Merge tables using all four join types and verify results with `df.shape` checks.

Agent-assisted wrangling:

- Direct a coding agent through the 8-step cleaning checklist. Verify each step, catch plausible-but-wrong cleaning decisions, and produce a reproducible cleaning log that fully documents every transformation.